APELEVEL / PUBLIC ENGINEERING RECORD



FULL DEVELOPMENT REPORT

Security, Wallets & Multiplayer Music

October 26, 2026 | Polygon-first authentication, wallet security, multiplayer music, and Tilt Court environment engineering

ENGINEERING TRACEABILITY

Repository: ragequitterseth-lgtm/ragequitterz-game

Working branch: feature/polygon-wallet-auth

Wallet-authentication milestone: **2601f6a**

Latest verified pushed music checkpoint: **b3b0259**

Status: development prototype; not a production release or security certification.

Prepared as a chronological account from the Zod dependency investigation through the final foreground-layer experiment. Findings marked “observed” come from the development record; hypotheses and incomplete validation are identified separately.

01 Executive summary

On October 26, ApeLevel advanced RAGE REALMZ through two significant, verified milestones: a Polygon-first wallet authentication workflow that preserves Ethereum-based NFT ownership checks, and a server-synchronized, expandable Ragehaven music system with an admin-only track-skip control. The work was not linear. It involved a dependency-security investigation, wallet-provider selection changes, a lengthy investigation of intermittent soundtrack failure, and repeated tests of disconnection behavior. The session closed with a separate, unresolved experiment to render trees, flags and pillars in front of player sprites.

VERIFIED RESULT

The final NPM installation reported **0 known vulnerabilities** in the local audit; a ten-track music collection was discovered successfully; the ApeLevel master-admin skip control was tested in the browser; ten manual skips and onward reshuffling were reported working; commit **b3b0259** was pushed to GitHub.

What changed

- Wallet sign-in requires Polygon Mainnet (chain ID 137); Ethereum RAGE QUITTERS NFT ownership is checked independently using read-only Ethereum contract calls.
- Browser wallet provider selection was moved to a standards-based injected-wallet chooser rather than relying on a single global wallet object.
- Ragehaven music progressed from five hardcoded tracks to automatic MP3 discovery with metadata-derived durations; ten actual files were recognized.
- The server stopped deleting the soundtrack timeline whenever the last WebSocket player briefly disconnected; browser reconnection and frequent music-state broadcast were added.
- The authenticated ApeLevel master-admin wallet gained a server-authorized SKIP TRACK action, avoiding full-song waits during testing.
- Environmental overhang work was attempted; fountain occlusion works, but the new tree/flag/pillar mask remains inaccurate and uncommitted.

Evidence boundaries

Successful manual skipping and loop-through do not independently prove long-duration, unattended playback, robust multi-client synchronization under varied network conditions, or exhaustive wallet and session-security testing. The observed disconnect causes were not fully isolated; codes 1001 and 1006 identify connection closure conditions, not their upstream root causes.

02 Chronology at a glance

Phase	Development evidence and outcome
Security audit	New WalletConnect/Reown dependency chain produced 11 high-severity audit findings. Older alternative produced 1 high and 8 moderate. Unused connector removed; subsequent audit: zero findings.
Wallet flow	EIP-6963-style wallet chooser, Polygon chain 137 login challenge, verified signature, Ethereum NFT read-only check.
Wallet checkpoint	Commit 2601f6a pushed on feature/polygon-wallet-auth; existing eight-page report captures that checkpoint.
Original soundtrack	Five hardcoded tracks; server-shared shuffled sequence, MP3 byte-range serving, client synchronization, ON/OFF control.
Observed failure	Songs ended at their reported duration and remained paused/ended rather than advancing; at times a new song began mid-track.
Diagnostics	Track/reset/disconnection logs showed zero-player resets and WebSocket close codes 1001 and 1006.
Recovery patches	Automatic browser reconnect, persistent server playlist across empty sessions, repeated realmMusic broadcasts.
Playlist expansion	music-metadata installed, filesystem scan and refresh added, 10 MP3s recognized with parsed durations.
Testing tool	Master-admin-only SKIP TRACK, server-side wallet permission check, UI shown only after authorized login.
Music checkpoint	Commit b3b0259 pushed to remote feature branch; 6 files changed, 484 insertions, 25 deletions.
Foreground experiment	Fountain occlusion working; trees/flags/pillars mask approximation looked wrong; work stopped uncommitted.

03 Initial Zod and WalletConnect security incident

Why the investigation began

The team considered adding a wallet connector to broaden the wallet login experience. Installing a WalletConnect provider pulled in dependencies associated with Reown and affected Zod versions. The resulting audit flagged multiple advisories. Several entries originated within overlapping transitive paths, so the number of findings was not equivalent to the number of independent exploited defects.

Initial results: 11 high-severity findings

Rather than using npm audit fix --force or indiscriminately upgrading packages, ApeLevel investigated the origin of the dependency chain. The intent was to avoid modifying a working game authentication stack in a way that could introduce API incompatibilities or wallet-signing instability. The original report records **11 high-severity audit findings** after installation of the proposed newer connector.

Alternative connector test: 1 high / 8 moderate

An older WalletConnect package was also evaluated. Although it sidestepped the particular Zod chain that triggered the initial high-severity cluster, it introduced its own affected WebSocket and URI-decoding packages. The test result was **1 high** and **8 moderate** findings. This did not represent a satisfactory improvement in the dependency posture.

Resolution: removal and native provider discovery

The unused WalletConnect integration was removed. RAGE REALMZ pursued installed browser-extension discovery using the EIP-6963 provider-announcement pattern, reducing reliance on an external connection package for this phase. The audit after uninstall reported **0 known vulnerabilities**. Later, after installing music-metadata, the user again observed an npm output reporting **0 vulnerabilities** across the then-installed dependencies.

SECURITY INTERPRETATION

A zero-finding npm audit only means the installed dependency graph did not match known advisories at that moment. It does not establish that application logic, signature verification, third-party extensions, WebSocket handling, or future token-spending flows are free of vulnerabilities.

04 Polygon-first identity, Ethereum ownership

Architectural decision

The user wanted the player-facing experience to operate on Polygon, anticipating Polygon-native TILT balances and possible future spending, without migrating the existing RAGE QUITTERS Ethereum NFTs. An EVM address derived from a conventional externally owned wallet may be used on both networks: proving control of the address and proving ownership of NFTs at that address are separate operations.

Sign-in workflow

- **01.** User chooses an injected EVM wallet. Browser evidence specifically showed MetaMask and Coinbase Wallet.
- **02.** Wallet connection is checked against Polygon Mainnet, chain ID 137.
- **03.** Server issues a nonce-bearing, short-lived SIWE-style login challenge.
- **04.** Browser signs with the selected wallet; server verifies signature and associated challenge properties.
- **05.** Server queries the Ethereum Mainnet NFT contract using read-only balanceOf and ownerOf operations.
- **06.** Server gates holder-only experience, character skins and separately authorized God Mode according to the authenticated session.

Important separations

A login signature is not an allowance or token-transfer approval. Ethereum ownership checks do not require switching the user wallet to Ethereum Mainnet and do not move tokens. Support for contract wallets and multiple wallet providers requires distinct validation, since signatures or address behavior can vary by network. Future TILT token spending on Polygon would require its own explicit transaction or approval flow and security review.

Contract and original checkpoint

The original report identifies the RAGE QUITTERS Ethereum collection contract as **0x70a95FbCe719d9B8C02DBe40C71fdDCc333a4e80**. The documented ownership test returned “Owns NFT: true” for the designated test address. Existing God Mode authentication also succeeded, but that is separate from proving all ordinary holder sign-in paths. Wallet milestone **2601f6a** was documented as pushed on the feature/polygon-wallet-auth branch; that earlier commit was not overwritten by the later music commit.

05 Multiplayer architecture and initial music design

RAGE REALMZ was already operating as a Node.js and WebSocket browser prototype. Players entered Ragehaven/Tilt Court with sprite animations, positional updates, world chat, portal checks and NFT/God Mode entitlements. The developer added a synchronized five-song soundtrack so participants in the same realm would hear the same musical sequence at approximately the same point in time.

Initial five-song manifest

Track	Declared runtime
all i know. (8bit).mp3	165,072 ms
are you in love_ (8-bit).mp3	164,424 ms
casper. (8-bit).mp3	152,832 ms
for you. (8-bit).mp3	217,800 ms
maybe I'm alone forever. (mystical 8 Bit).mp3	204,024 ms

Server timeline and shuffle

The realm-music.js module used cryptographically sourced random integer selection for a Fisher-Yates shuffle. A session stored the track order, index, and startedAt timestamp. On each update, the server compared Date.now() with declared track duration to advance the active session. After exhausting the shuffled order, it generated a new order while avoiding an immediate repetition of the final song. realmMusic packets included realm, filename, startedAt, serverTime, and duration.

Browser playback

The client maintained an HTML Audio object and selected the current file by realm/filename. Its synchronization routine estimated playback position from serverTime minus startedAt plus elapsed local monotonic time. It preloaded the next source when it received a new session key, sought if needed, and exposed a local MUSIC ON/OFF toggle. The server also provided MP3 byte-range (HTTP 206 Partial Content) support to facilitate browser seeking.

Design objective

Unlike local randomized playlists, the server-owned approach allows multiple players to share a track sequence and recover approximately matching playback positions when joining after a song starts. A key challenge is distinguishing a server timeline reset from a missed client state update or dropped network connection.

06 The music stopping bug: first symptoms

Observed end-of-song silence

The reported failure was repeatable: after one or more songs, the soundtrack ended and did not automatically begin another. A browser monitor recorded a full-length playback of “maybe I’m alone forever. (mystical 8 Bit).mp3” reaching roughly **204 / 204 seconds**, then remaining **paused: true**, **ended: true**, and **error: none** for more than thirty seconds. This was stronger evidence of a track-transition/state problem than a corrupt MP3.

Early client recovery attempts

A 5-second synchronization interval was enabled on the client and an audio ended event handler was added. The code also included a guard to avoid re-playing stale music messages after the expected song had finished. These were defensible recovery ideas, but they did **not** fix the observed track-change failure. In particular, a stale latestMusic packet could not meaningfully advance playback if the browser never received an updated session.

First server logging

The server began logging [REALM MUSIC] with the realm, track filename, player count, and startedAt timestamp when a song was announced. An early log showed the same song starting at timestamps separated by about 70 seconds. That suggested a playlist reinitialization, but the evidence alone did not prove it had reset while an active player remained connected: browser refreshes or server restarts could also account for it.

Failed patch and why it mattered

An attempt to alter the broadcast frequency failed with “STOP: Music broadcast section not found.” Because the file was not changed, that attempted patch was **not** counted as installed. Subsequent work used the exact source snippet returned by grep, avoiding assumptions about whitespace and formatting.

07 WebSocket investigation and playlist resets

Reset logging established a zero-player condition

The diagnostic server output recorded [REALM MUSIC] followed by [MUSIC RESET] ragehaven player count reached zero. This proved that, at the reset moment, the server believed no player occupied the hub. It did not by itself identify why the player disappeared.

Disconnection handler instrumented

The server’s WebSocket close handler was changed to log player ID, realm, close code, reason, and remaining player count. When tested, the output reported close code **1006** with remainingPlayers: 0, followed by [MUSIC RESET]. Later logs included code **1001** and another 1006. Code 1006 represents abnormal closure without a completed close handshake; 1001 generally indicates going away. Neither code alone establishes whether a browser tab, network, reverse proxy, refresh, or development environment initiated the change.

Original browser behavior

The browser’s close handler had only updated the status indicator and displayed “Connection lost. Refresh to reconnect.” It did not retry automatically. This created a poor recovery path in the Codespaces-hosted prototype and made brief disconnects more disruptive.

Reconnection patch

The client was updated to retain the player nickname and schedule a retry after approximately three seconds when its prior socket closed. A socket-identity and readyState check was intended to avoid competing retries. The user later observed the game displayed Connected and heard active music, although this observation alone was not a controlled server-stop/restart proof of reconnection behavior. Authentication permissions on a new socket should be freshly verified; a reconnected nickname is not a substitute for wallet authorization.

Critical server defect

The server cleared the entire realm music session and deleted lastMusicTrack every second that the player count was zero. Thus even a momentary absence of players could destroy the shared timeline; a rejoining player would begin at a newly shuffled track, explaining at least some apparent mid-song changes.

08 Corrective music patches and verification

Persisting music across temporary absence

Once the precise server lines were inspected, the empty-realm branch was changed to **continue without clearing the session**. The session and its startedAt timestamp remain stored while nobody is online. On re-entry, getRealmMusic catches up through any elapsed durations and returns the appropriate track. The trade-off is deliberate: the public-hub soundtrack remains conceptually running even when the hub is empty.

Periodic current-state broadcasts

Previously a packet was sent primarily on track changes. The interval was changed to broadcast the current realmMusic state every second while at least one player was in Ragehaven. Repeated packets with an unchanged session key allow browsers to correct drift and reduce the impact of an isolated missed track-change message. This increases small WebSocket packet traffic but avoids depending on a single transition notification.

Observed outcome at this stage

Music resumed in the prototype and later manual skip tests worked. However, the user initially reported that a song still stopped even while the browser said Connected, then reported a new song starting partway through another track after connection churn. The investigation ultimately isolated **confirmed timeline resets** as one real contributor; not every underlying disconnection trigger was independently reproduced or conclusively assigned.

Remaining playback edge cases

- Test 30–60 minutes of unattended playback and collect timestamped [REALM MUSIC] output at natural song boundaries.
- Simulate brief disconnection, reconnect, full reload, network stall, and joining a second browser mid-track.
- Confirm server and browser clocks/offset handling, packet arrival order, file loading, and mobile browser autoplay restrictions.
- Assess volume normalization, track-metadata correctness, and whether playback state survives realm transitions as intended.

09 Expanding beyond five songs

Why the playlist needed redesign

The original manifest was a manual list of five filenames and durations. The user explicitly required it to remain open-ended as new MP3s were uploaded rather than being limited to the initial set. The local package music-metadata was installed, after which npm reported 14 packages added and zero known vulnerabilities.

Automatic discovery design

The replacement realm-music.js scans the Ragehaven music directory, filters MP3 filenames, checks each file’s size and modification time, and uses music-metadata parseFile to determine duration in milliseconds. A cached signature avoids re-reading unchanged files. It scans on startup and schedules subsequent checks every 30 seconds; scan errors and skipped tracks are logged. New files are appended into the remaining shuffled sequence while attempting to preserve the current track.

The ten files actually detected

MP3 file	Parsed duration
I Want Power (1).mp3	210,000 ms
a menace now. (8-bit).mp3	157,800 ms
all i know. (8bit).mp3	165,072 ms
are you in love_ (8-bit).mp3	164,424 ms
casper. (8-bit).mp3	152,832 ms
cotton candy. (8bit).mp3	136,440 ms
for you. (8-bit).mp3	217,800 ms
maybe I'm alone forever. (mystical 8 Bit).mp3	204,024 ms
right mind (8 bit).mp3	193,464 ms
swallowed. (8 - bit).mp3	69,744 ms

The import check produced ten [MUSIC DISCOVERED] log entries and “PASS: Music system loaded”. The runtime of the initial recognized collection is approximately 28 minutes and 39 seconds. This establishes detection and metadata parsing, not an indefinite guarantee of media playback for any future uploaded MP3.



Implementation caveat

Playlist membership is refreshed without restarting the process, but file replacement/removal while a track is active deserves extra testing. The code preserves the playing file when it remains available, while the new catalogue can alter the queue of pending tracks. Operational limits depend on filesystem, file validity, and available server resources; “unlimited” means no fixed five-song hardcode, not literally unbounded capacity.

10 ApeLevel-only SKIP TRACK testing control

Requirement

Waiting several minutes for every natural track transition made testing inefficient. The user requested a skip button visible only to the ApeLevel God Mode wallet. Importantly, clicking the button was intended to change the shared soundtrack for everyone in the realm, not merely jump the administrator’s local audio player.

Music engine

A skipRealmMusic(realm) function was appended to the soundtrack module. It advances the session order, starts the new active track at Date.now(), and returns an updated realmMusic packet. The code includes a check to avoid immediately replaying the previous track when possible. A single-track collection restarts the lone track.

Authorization boundary

The WebSocket message handler accepts a skipRealmMusic command only when the socket already holds an authenticated wallet and its God Mode record has the master-admin role, combined with the existing isMasterAdmin predicate. The server rejects all other requests with musicSkipDenied. This server-side check is essential; a hidden browser button alone cannot enforce permission.

Browser interface and authentication result

The successful authSuccess response was extended with canSkipMusic computed by the server from its own authorization checks. The game creates a hidden SKIP TRACK button beside MUSIC ON/OFF, reveals it only after that permission appears, and hides it on failed authentication, wallet disconnect, or new connection. The button sends a skipRealmMusic request through the existing WebSocket transport. Skipping is not a token transaction and needs no fresh wallet signature each time, but the current authenticated WebSocket session remains required.

Observed test

The user reported the button appeared and worked. The follow-up test instructed ten skips to complete the initial ten-song cycle and verify the next shuffle. The user replied “works.” This is the session’s strongest positive evidence for manual playlist progression across an entire shuffle and into the next sequence. It is not the same as a ten-track unattended playback test.

11 Git commits and remote proof

Two distinguishable milestones

Commit / branch	What the record establishes
2601f6a / feature/polygon-wallet-auth	Earlier Polygon-first wallet authentication milestone, documented in the initial Oct 26 public report. Six files; 175 insertions and 13 deletions according to that original report.
b3b0259 / feature/polygon-wallet-auth	Music system checkpoint: “Add expandable synchronized realm music and God Mode track skipping”. Git reported 6 files changed, 484 insertions, 25 deletions.

Exact commit confirmation

LOCAL COMMIT OUTPUT

[feature/polygon-wallet-auth b3b0259] Add expandable synchronized realm music and God Mode track skipping

6 files changed, 484 insertions(+), 25 deletions(-)

Exact push confirmation

REMOTE PUSH OUTPUT

To https://github.com/ragequitterseth-lgtm/ragequitterz-game

2601f6a..b3b0259 feature/polygon-wallet-auth -> feature/polygon-wallet-auth

branch feature/polygon-wallet-auth set up to track origin/feature/polygon-wallet-auth

The remote push output is evidence that GitHub accepted the update of the named feature branch to b3b0259. It does not prove a deployment, a production release, a merge to development, or exhaustive correctness. The user was explicitly advised that the later, unverified environment-layer changes were **not** included in this checkpoint.

Source control scope

The commit command staged server.js, realm-music.js, public/game.js, public/realm-audio.js, package.json, and package-lock.json; Git reported six files changed. The MP3 assets themselves were not staged by that command. Newly uploaded audio files and experimental map masks must be tracked or deployed separately if they are to appear in another environment.

12 Tilt Court foreground and depth sorting: unfinished

Original goal

The user wanted sprites to move behind tall environment objects—trees, flags, pillars, and architectural decorations—and remain visible when walking in front of them, consistent with classic top-down role-playing games. The central purple crystal fountain already displayed acceptable foreground occlusion and was expressly not the target for further changes.

Existing map and rendering

Tilt Court artwork lives in public/assets/maps/ragehaven/tilt-court/tilt_court_layer_package_v2/. The package includes 01_existing_hub_terrain.png, 02_jungle_gym_beanstalk_route.png, 03_foreground_overhangs.png, map references, collision mask and 614,400-byte collision binary. The three main PNG layers measure 1448 by 1086 and are scaled into a 960 by 640 canvas. The README describes the first image as plaza floor and the second as the crystal, stairs and environment, and notes that the tighter composition does not actually include the full beanstalk route.

Existing fountain occlusion

The game loads the transparent fountain PNG separately and renders it above the character pass. An earlier y=405 single-depth heuristic had already been adjusted. The user confirmed “the fountain works great,” so there is no reason to regress that working treatment while resolving other environment sprites.

Syntax incident and recovery

An intermediate patch accidentally merged a JavaScript comment with the defaultGorillaAnimations conditional and produced “SyntaxError: Unexpected token else” at public/game.js line 449. The first exact string replacement failed due to whitespace. A second regular-expression-based replacement successfully restored the conditional, and the user reported “pass.” This emphasizes the importance of running node --check after automated source transformations and verifying the rendered game rather than trusting success text printed before syntax checks.

Image inspection

Inspection showed Layers 01 and 02 use indexed-colour PNG mode P; 03 uses RGBA. Layer 02’s RGBA conversion had a 0-to-254 alpha range, approximately 235,504 fully transparent pixels and 1,337,024 partially transparent pixels. Thus transparent compositing is available, but alpha alone does not identify which pixels are tree crowns, flags, columns, or walkable foreground boundaries.

Failed experimental mask

A Python/Pillow script created 04_environment_foreground.png by cutting large polygon regions from Layer 02, then drawing the result over every player after the already-working fountain. A screenshot showed a result the user described as “totally off.” This was not successful fine-grained RPG occlusion: broad polygon masks can duplicate or cover unrelated terrain and do not account for each object’s footpoint/depth. The session ended here, without committing these environmental changes.

13 Known limitations, next engineering work

Priority A — foreground repair

- Do not modify the working fountain foreground. Remove/disable the inaccurate 04_environment_foreground overlay while replacing it.
- Create accurate transparent sprites/masks for each tall object, rather than eight rough regions covering large portions of the map.
- Associate each tree, flag, and pillar with a world-space footpoint or depth threshold; draw objects before or after each character based on relative y/depth and overlap.
- Validate near/far positions for each object with a named test matrix, including multiple player sprites, animations, portal graphics, and usernames.
- Keep the collision binary and camera positions unchanged unless a separate confirmed collision issue requires correction.

Priority B — music regression suite

- Leave the game idle for several complete natural playlist cycles without invoking SKIP TRACK; compare browser state with server transition logs.
- Test two simultaneous browsers, joining mid-track, switching realms, refreshing, server restart, and deliberate network interruption.
- Capture whether reconnection keeps the local music preference and requires appropriate wallet reauthentication for restricted actions.
- Test MP3 additions, edits and removals while the server is running. Confirm no missing-file loop and that newly discovered tracks participate in reshuffles.

Priority C — wallet and access hardening

- Invalidate session authorization appropriately on wallet account/chain changes, explicit disconnect, expired challenges, and new socket connections.
- Exercise negative tests: forged skip packets, unauthenticated clients, holders who are not master-admins, and authorization after reconnection.
- Continue testing MetaMask, Coinbase Wallet and additional EIP-6963 providers; separately assess contract-wallet behavior and mobile support.
- Display Polygon TILT balance read-only first; keep any future token spending behind a separate explicit authorization and threat review.

Priority D — foundational gameplay

- Develop the player-following camera over shared world coordinates, preserving independent local viewports and server-authoritative positions.



- Implement the zero-to-100 Rage Bar and realm ejection, NFT-linked player statistics, and wider Ragehaven expansion.

14 Final findings and public development status

COMPLETED AND PUSHED

Polygon-first wallet sign-in and Ethereum NFT checks; dependency audit resolution; expandable ten-track music discovery; persistent shared music timeline; repeat soundtrack broadcasts; browser reconnection attempts; server-authorized ApeLevel master-admin SKIP TRACK; GitHub push to **b3b0259**.

OBSERVED BUT NOT FULLY REGRESSION-TESTED

The user confirmed admin skips across ten tracks and onward reshuffling. Natural unattended transitions, disconnect root cause isolation, full reauthentication lifecycle, and all multiuser edge cases remain open for structured testing.

NOT FINISHED / NOT IN LAST PUSHED CHECKPOINT

Proper tree, flag, and pillar depth occlusion. The first 04_environment_foreground.png mask was visually rejected; working fountain coverage must be preserved.

Why this development session matters

ApeLevel did more than add features. The team traced unexpected behavior through browser playback state, network disconnect signals, server session lifecycle, and server-to-client synchronization timing; revised the actual failure mode rather than relying exclusively on client workarounds; reduced an avoidable dependency risk; and added a test mechanism restricted to an authenticated administrative wallet. The remaining defects and uncertainties are recorded rather than obscured.

Supporting record and provenance

This report is based on the uploaded eight-page “RAGE REALMZ Public Development Report” dated October 26, 2026 and the in-conversation Codespaces commands, console excerpts, code snippets, terminal outputs, Git commit/push text, and final screenshots. The underlying Git repository was not independently cloned or audited in preparing this report. Quoted terminal outputs are supplied evidence; all other architectural descriptions summarize the provided materials. This report is a technical development narrative, not a third-party security certification or release attestation.

Repository: https://github.com/ragequitterseth-lgtm/ragequitterz-game

Milestone commit: b3b0259 | Branch: feature/polygon-wallet-auth | Report date: October 26, 2026